Universidad de La Laguna

Continuous Integration (CI), Vite, Github Actions and Github pages.

Continuous Integration (CI), Vite, Github Actions and Github pages.

Link to code examples in these slides

Authors

Marco Pérez Padilla

marco.perez.padilla.34@ull.edu.es

Pablo Aswani García

pablo.aswani.16@ull.edu.es

Iván Luis Estévez

ivan.luis.estevez.24@ull.edu.es

Index

- Continuous integration
- Vite
- GitHub Actions
- Continuous delivery
- Continuous deployment
- Github pages
- Monorepos
- CI/CD Alternatives

What is Continuous integration?

Let’s see an example without continuous integration

What is Continuous integration?

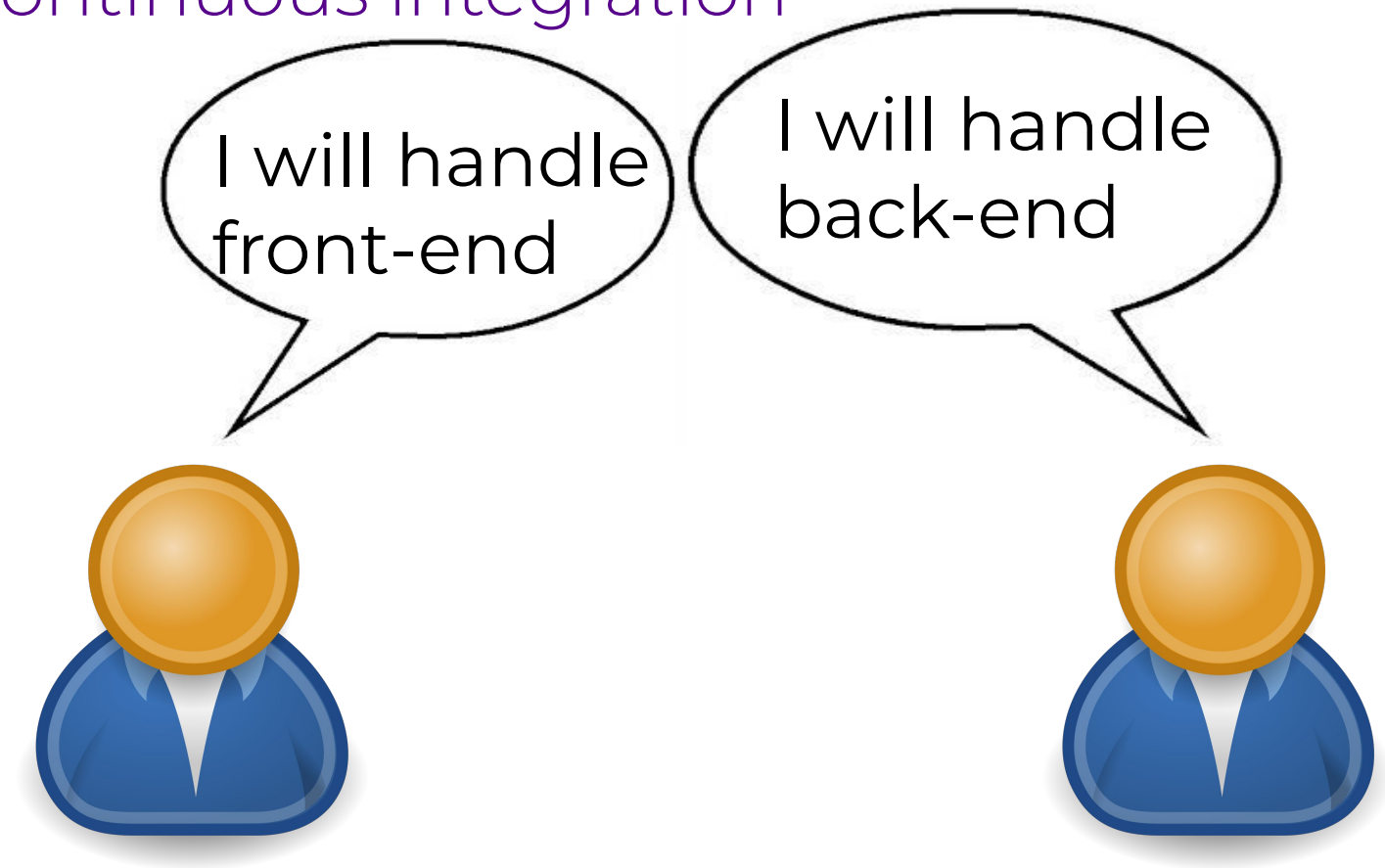
Let's see an example without continuous integration

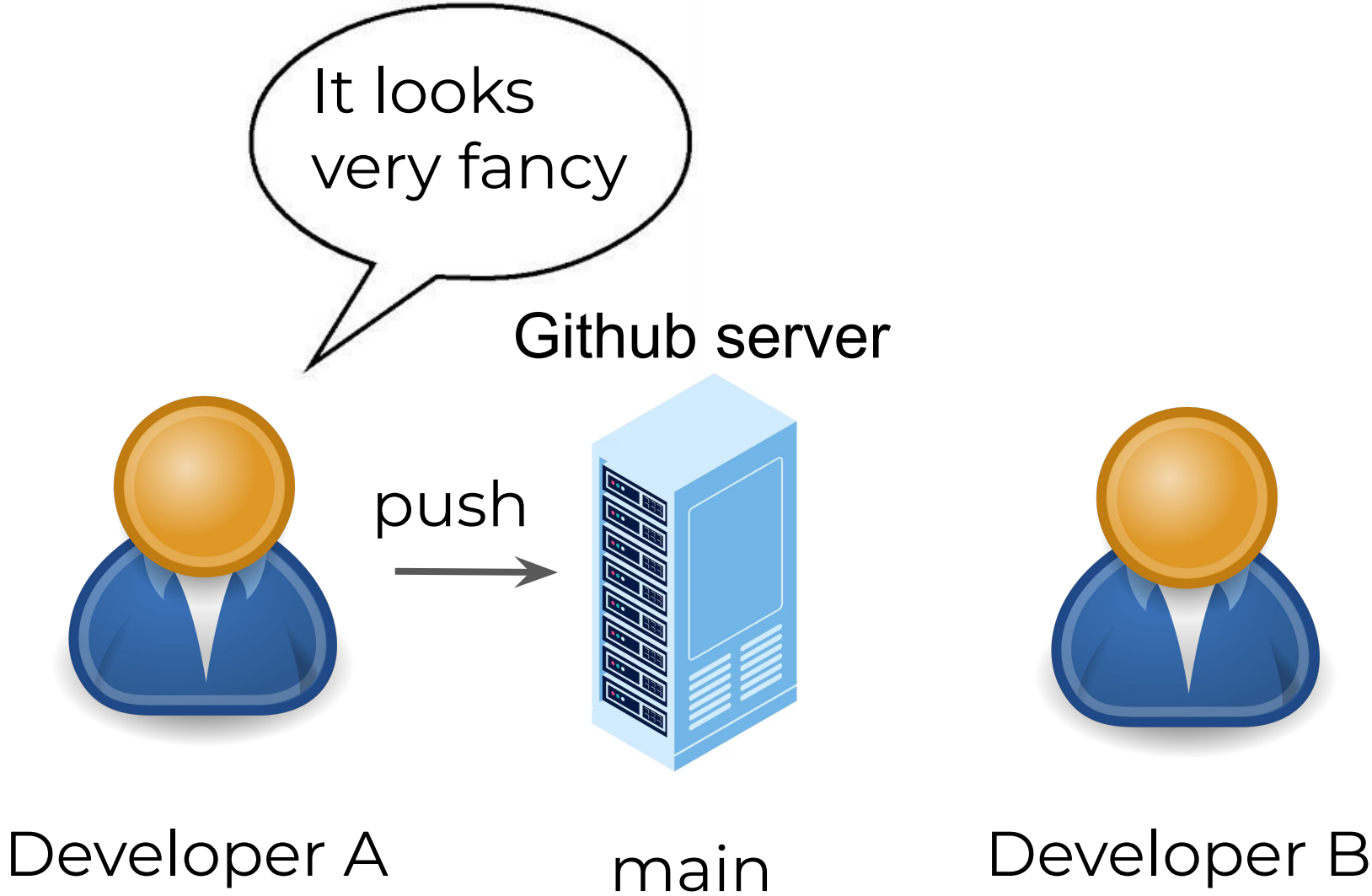
Developer A

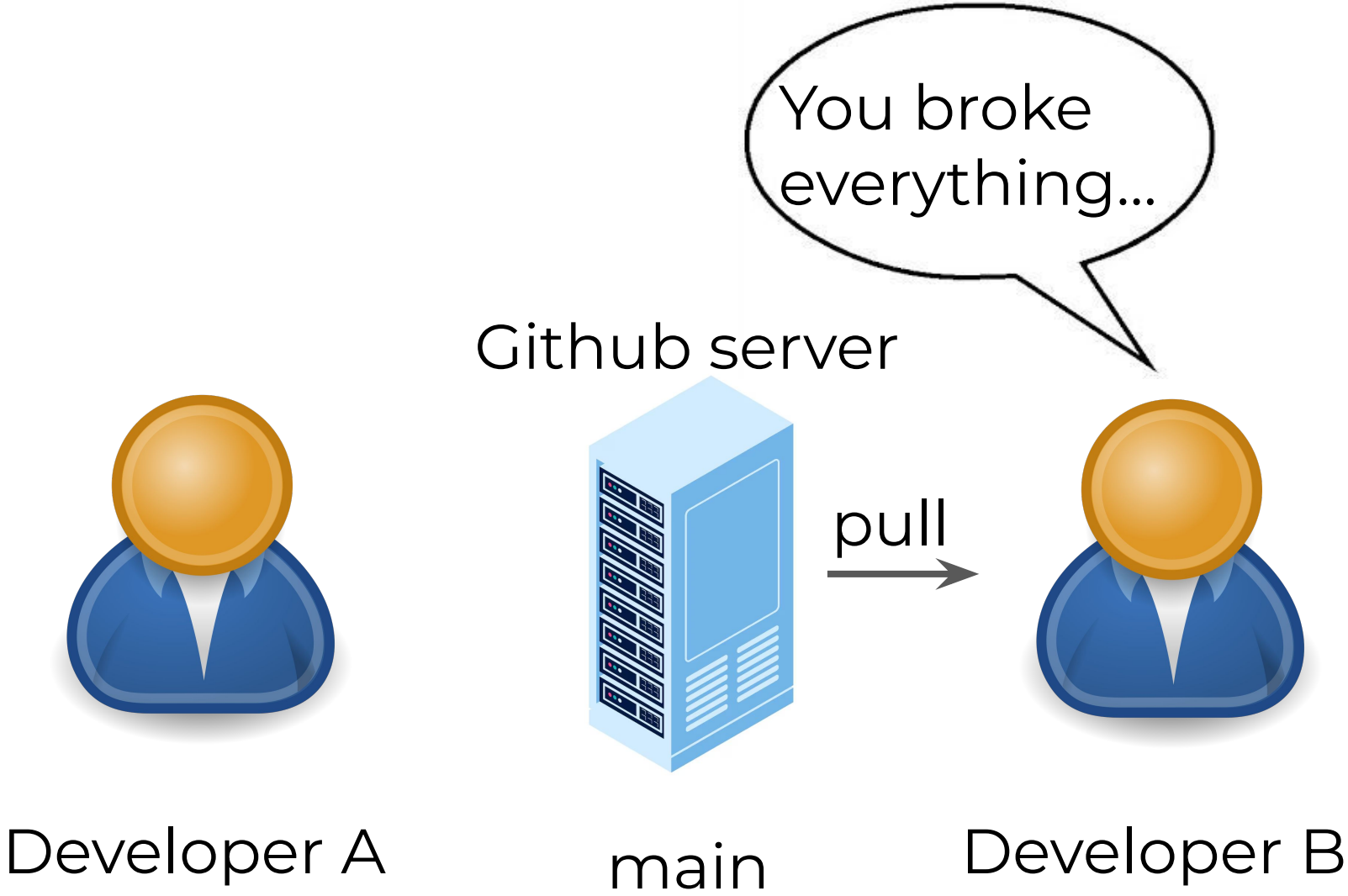
Developer B

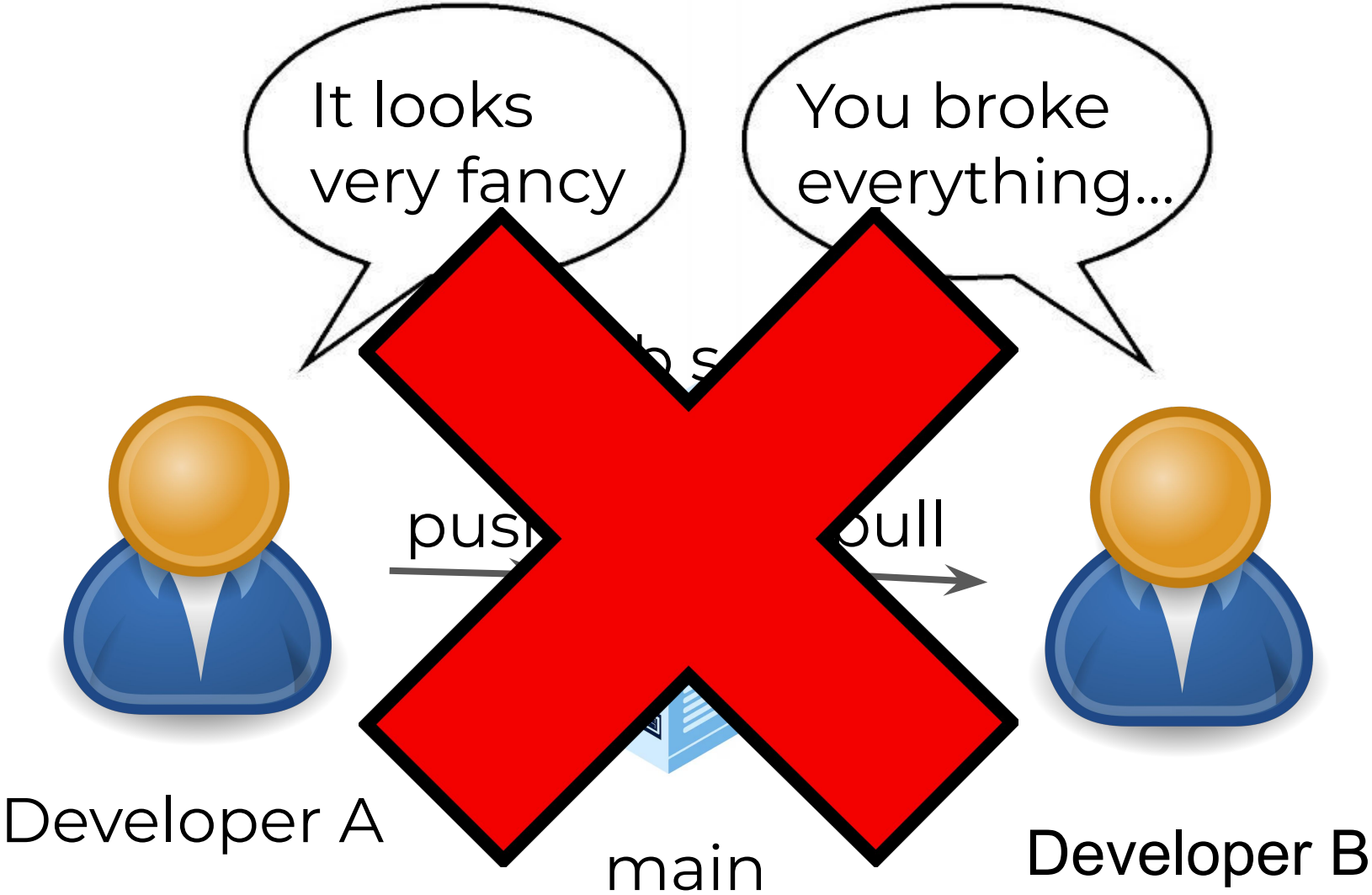
What is Continuous integration?

Let's see an example without continuous integration




It looks very fancy
Github server
push
Developer A
main
Developer B


You broke everything...
Github server
pull
Developer A
main
Developer B


It looks very fancy
You broke everything...
Developer A
main
Developer B

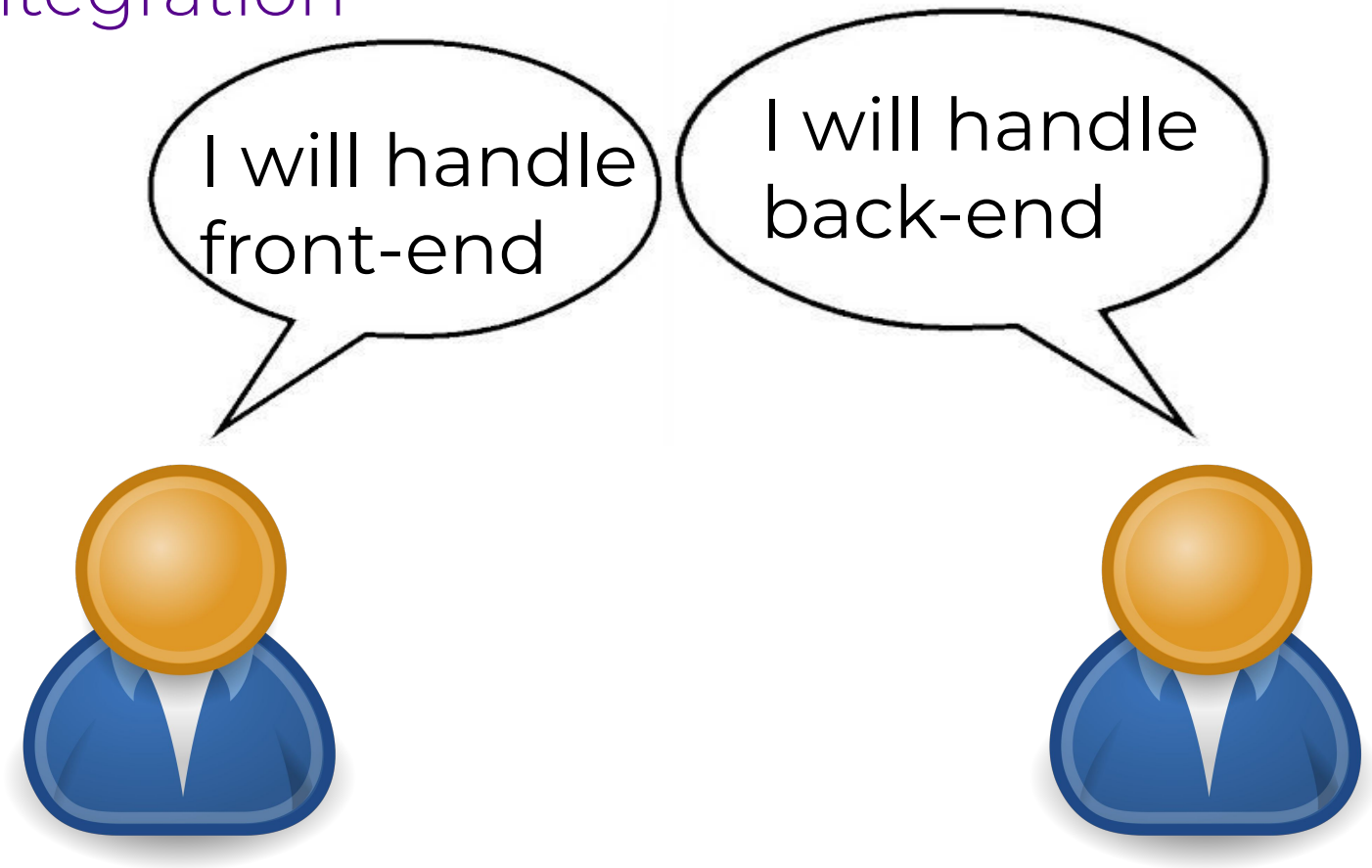
How do we avoid that?

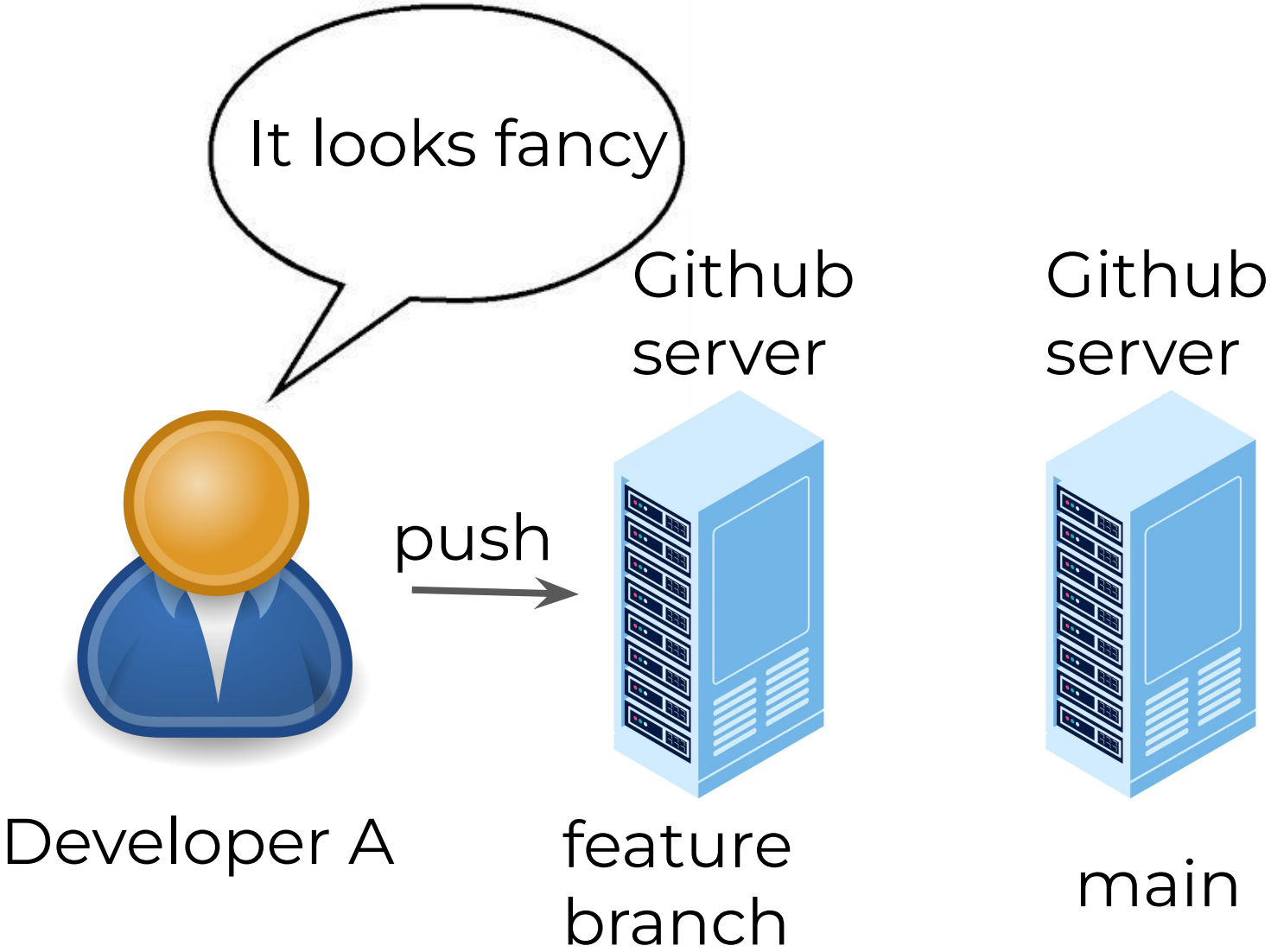
How do we avoid that?

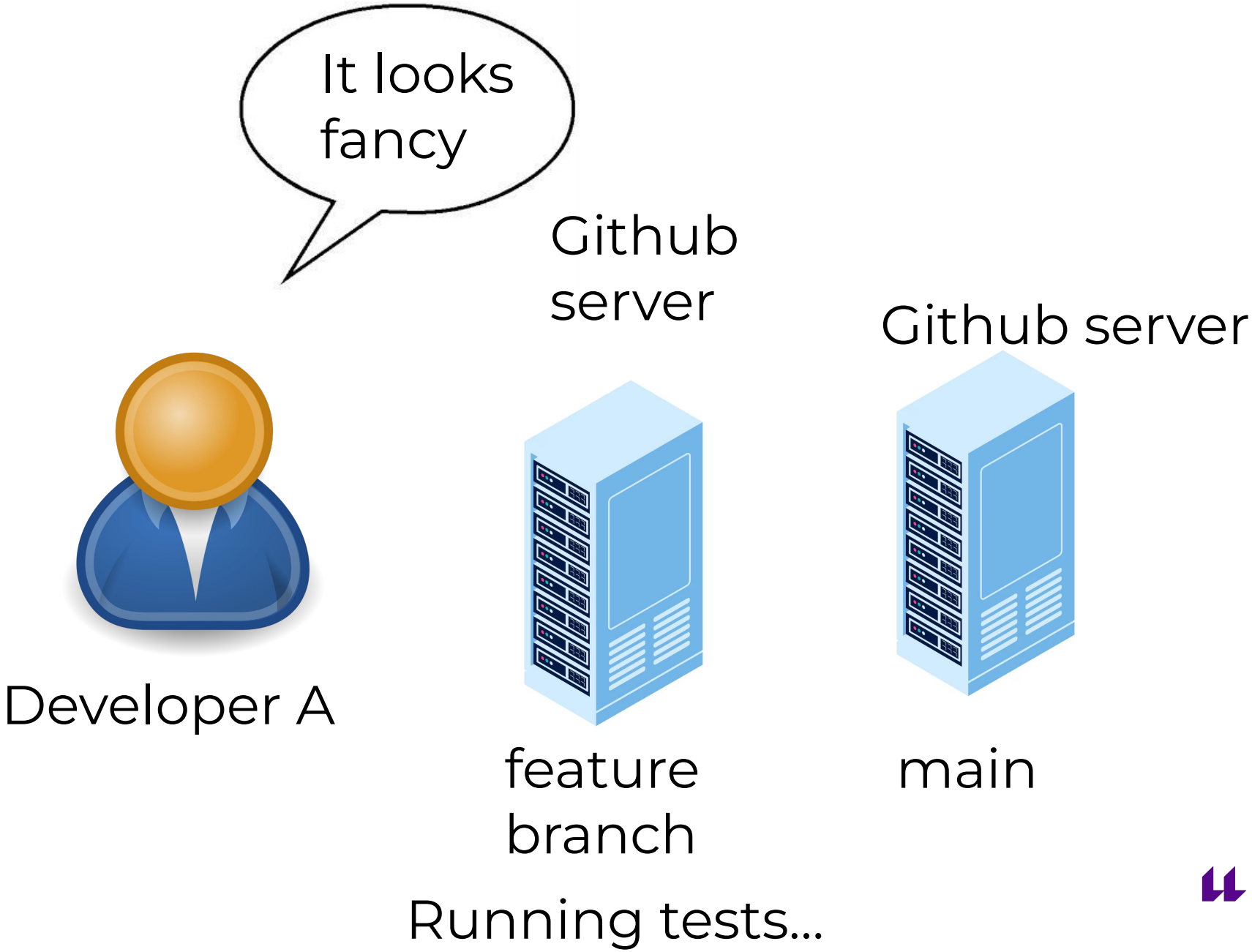
Continuous integration

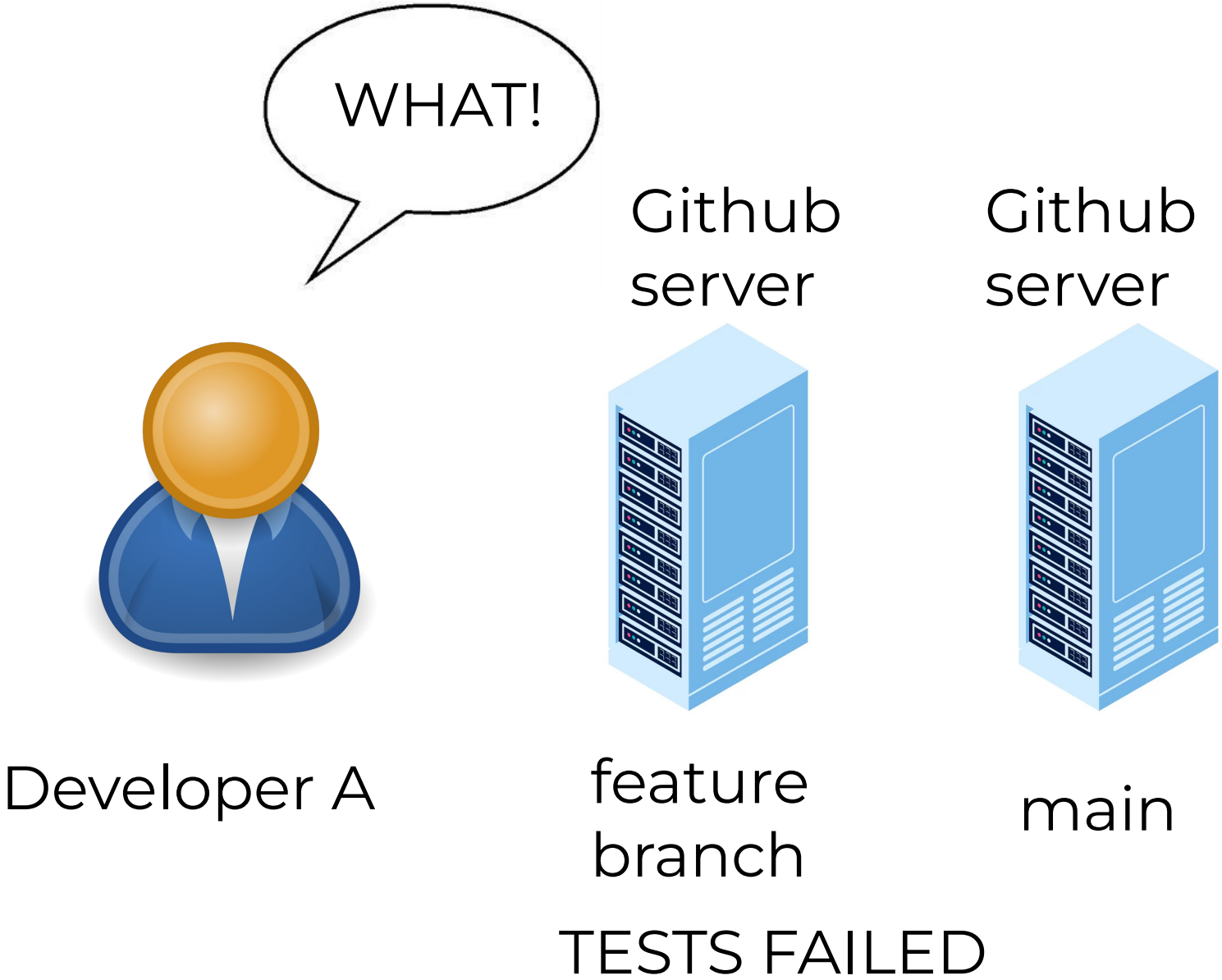
What is Continuous integration?

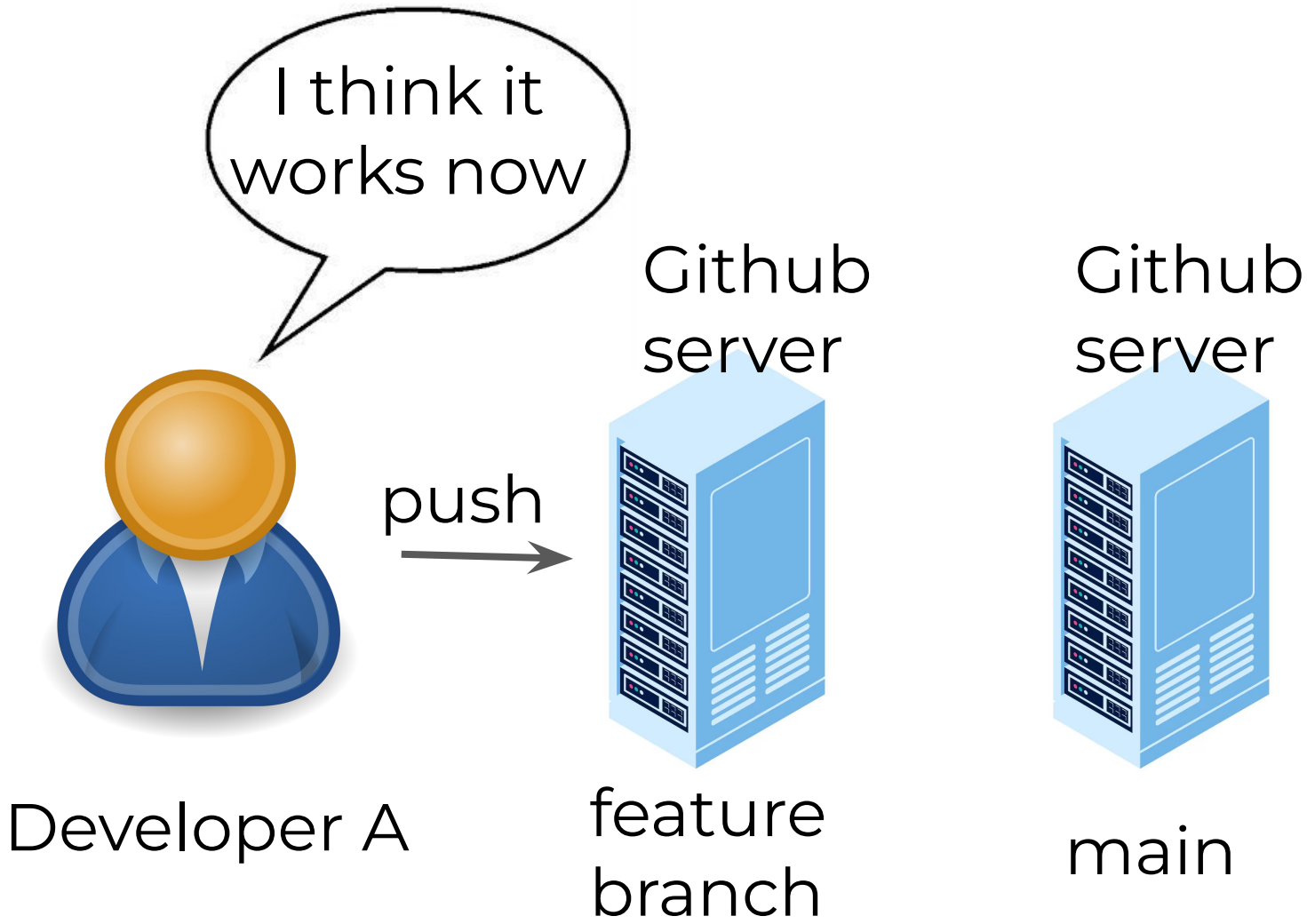
Let's see an example with continuous integration




It looks fancy
Github server
Github server
push
Developer A
feature branch
main


It looks fancy
Github server
Github server
Developer A
feature branch
main
Running tests...

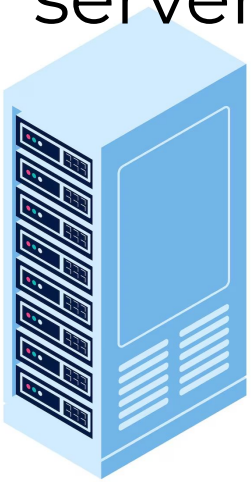

WHAT!
Github server
Github server
Developer A
feature branch
main
TESTS FAILED



Running tests...

Developer A

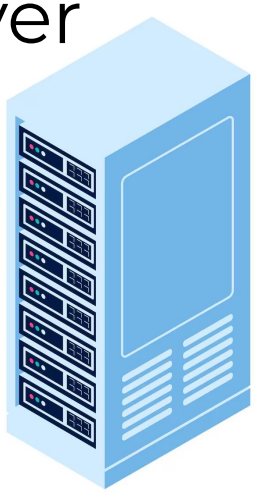
Github server



feature branch

merge

Github server



main

Tests succesful

Developer A

Developer B

Vite

- Modern frontend build tool
- Focused on **speed and simplicity**
- Great fit for TypeScript projects

Why use Vite?

Problems with Traditional Tools:

Why use Vite?

Problems with Traditional Tools:

- Slow startup time

Why use Vite?

Problems with Traditional Tools:

- Slow startup time
- Full rebundling on every change

Why use Vite?

Problems with Traditional Tools:

- Slow startup time
- Full rebundling on every change
- Poor scalability in large projects

Key Features of Vite

- Instant server start

Key Features of Vite

- Instant server start
- Fast Hot Module Replacement (HMR)

Key Features of Vite

- Instant server start
- Fast Hot Module Replacement (HMR)
- Native ES Modules support

Key Features of Vite

- Instant server start
- Fast Hot Module Replacement (HMR)
- Native ES Modules support
- Optimized production build

Key Features of Vite

- Instant server start
- Fast Hot Module Replacement (HMR)
- Native ES Modules support
- Optimized production build
- Simple and minimal configuration

How to install

```
$ npm create vite@latest
$ npm install
$ npm run dev
```

Optional configuration file: `vite.config.ts`

The `index.html` file must be in the root of the project.

Github Actions

- Automation
- Integrated
- Event-driven

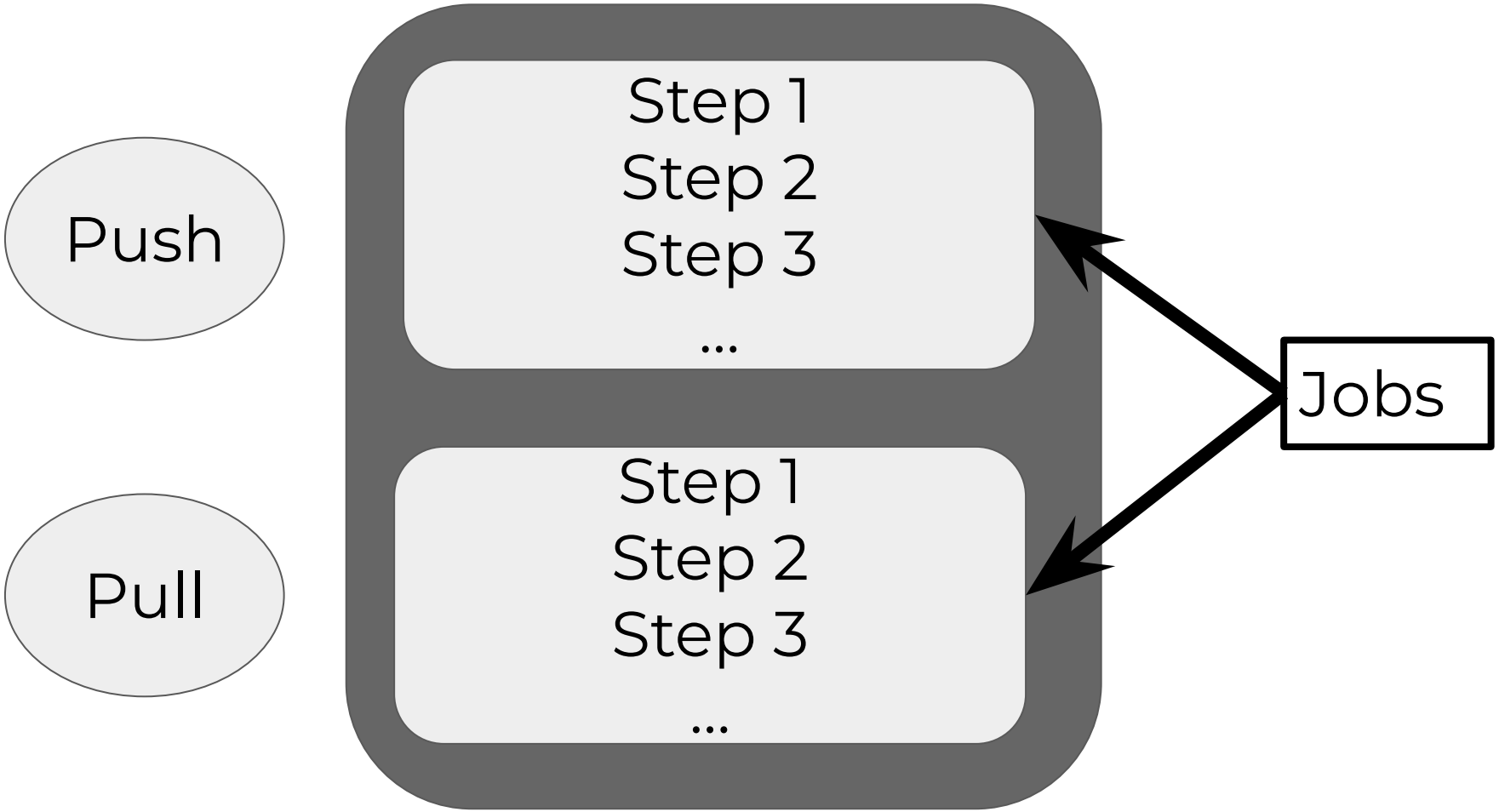
Github actions

- To use GitHub Actions, you need one or more YAML files.

EVERY `.yml` is executed automatically inside `.github/workflows`

- triggered by certain events(pull, push, etc...)

Github actions



Github actions

What do we want to add inside `.github/workflows`?

We are going to create a workflow that:

Github actions

What do we want to add inside `.github/workflows`?

We are going to create a workflow that:

- Automatically runs our code

Github actions

What do we want to add inside `.github/workflows`?

We are going to create a workflow that:

- Automatically runs our code
- Installs project dependencies

Github actions

What do we want to add inside `.github/workflows`?

We are going to create a workflow that:

- Automatically runs our code
- Installs project dependencies
- Executes tests

Github actions

What do we want to add inside `.github/workflows`?

We are going to create a workflow that:

- Automatically runs our code
- Installs project dependencies
- Executes tests
- Detects errors before merging

General structure of a workflow

Main elements:

- name
- on
- jobs
- uses
- runs-on
- steps

```
name: CI Pipeline
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - run: npm install
       - run: npm test
```

“main” and “on” elements

`name`: Name of the workflow

`on`: When the workflow runs:

- `push`
- `pull_request`
- `workflow_dispatch`

“jobs” and “uses” elements

`jobs`: Defines the set of independent or interconnected tasks.

`uses`: Runs a pre-made action created by GitHub, the community, or your team.

“runs-on” and “steps” elements

`runs-on`: Where the job runs (Ubuntu, Windows, or macOS)

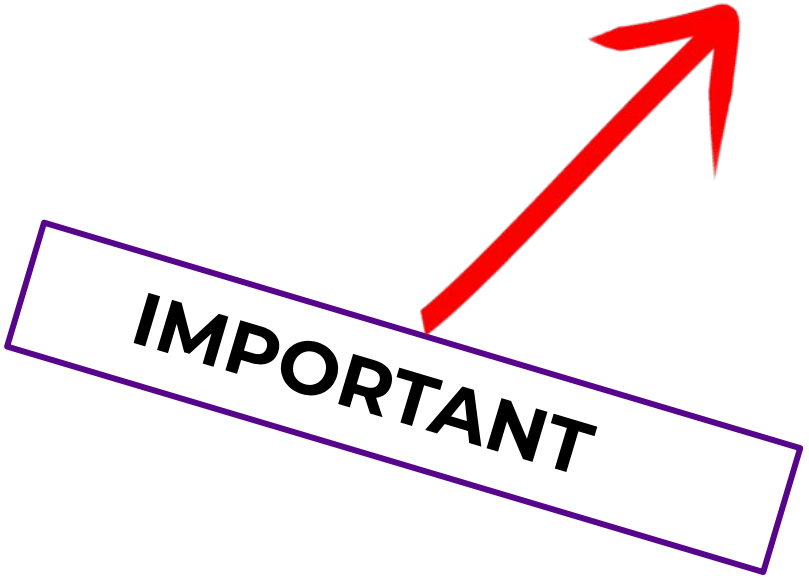
`steps`: Defines the sequence of individual commands or actions that execute in order within a job.

Github Actions Examples:
github-actions-fundamentals

Workflow

Let's start by making a feature branch

```
$ git checkout -b feature/ci-cd-pipeline
```



Continuous Delivery

- Code is always in a deployable state

Continuous Delivery

- Code is always in a deployable state
- Build, test, deploy to staging (all automated)

Continuous Delivery

- Code is always in a deployable state
- Build, test, deploy to staging (all automated)
- Production deployment requires a human click

Continuous Delivery

- Code is always in a deployable state
- Build, test, deploy to staging (all automated)
- Production deployment requires a human click
- QA team validates before going live

Continuous Deployment

- Ship features to users as fast as possible

Continuous Deployment

- Ship features to users as fast as possible
- 100% automated - no manual steps

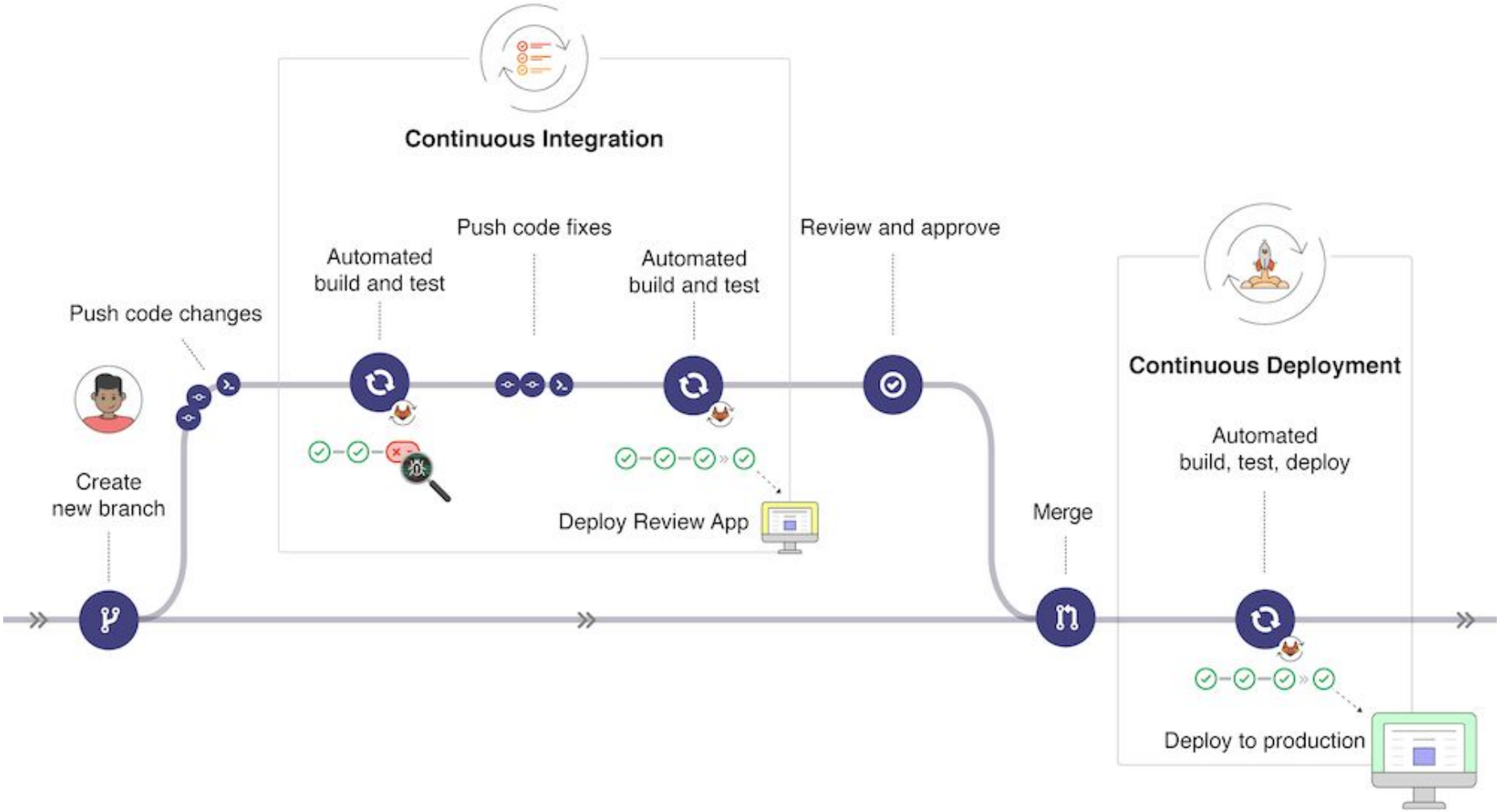
Continuous Deployment

- Ship features to users as fast as possible
- 100% automated - no manual steps
- Production deployment happens automatically

Continuous Deployment

- Ship features to users as fast as possible
- 100% automated - no manual steps
- Production deployment happens automatically
- Users get updates immediately

Continuous Deployment



Github pages

- Some key characteristics:
  - Static website hosting service

Github pages

- Some key characteristics:
  - Static website hosting service
  - Directly from a GitHub repository

Github pages

- Some key characteristics:
  - Static website hosting service
  - Directly from a GitHub repository
  - Free and easy to use

Github pages

- How does it work?
  - Push code to repository
  - GitHub selects a branch (usually main or gh-pages)
  - Static files are served as a website

Github pages

- How does it work?
  - Push code to repository
  - GitHub selects a branch (usually main or gh-pages)
  - Static files are served as a website

- Deployment process with Vite
  - Build project locally (`npm run build`)
  - Generate static files (dist/)
  - Upload files to repository branch (gh-pages)

Github pages

- Github Pages are usually combined with Github Actions, allowing automatic build and/or deployment.

Github pages

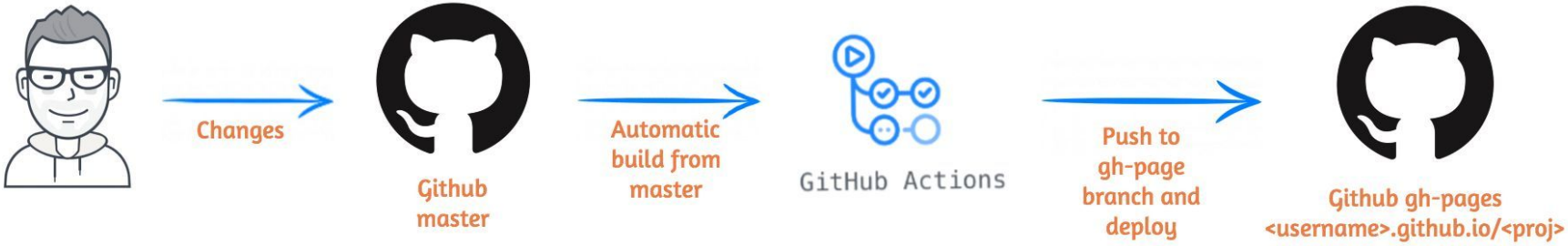
- Github Pages are usually combined with Github Actions, allowing automatic build and/or deployment.

- Key idea
  - GitHub Pages = hosting service
  - GitHub Actions = automation tool

Github pages

- Github Pages are usually combined with Github Actions, allowing automatic build and/or deployment.

- Key idea
  - GitHub Pages = hosting service
  - GitHub Actions = automation tool



Monorepos

- A monorepo combines multiple projects or components into a single shared repository

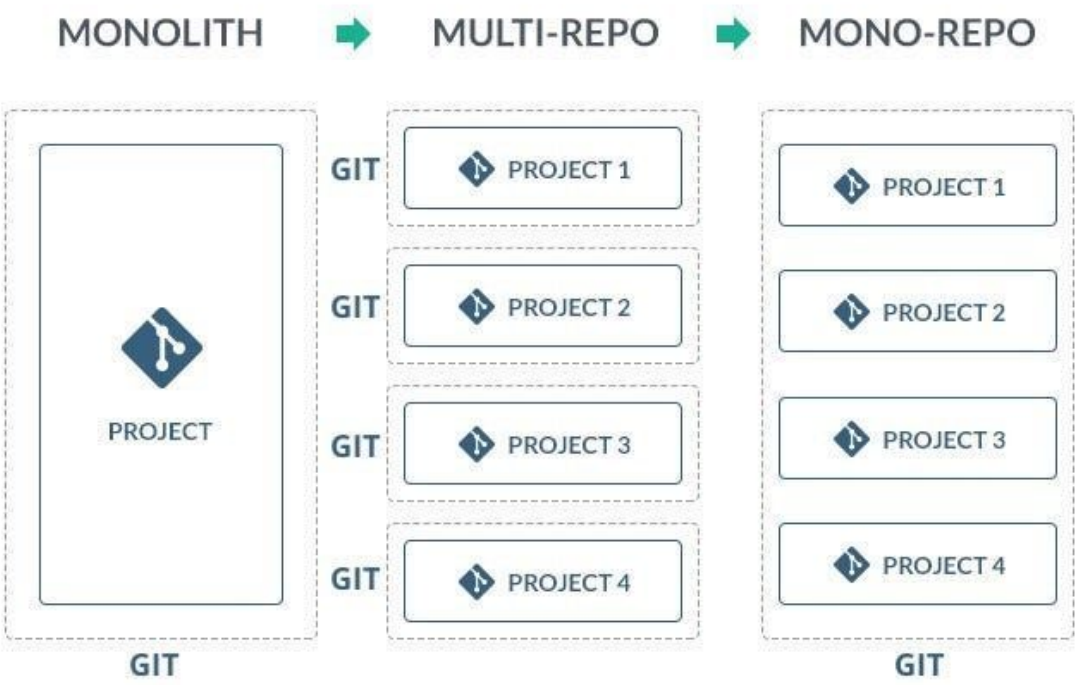
Monorepos

- A monorepo combines multiple projects or components into a single shared repository
- A Multi-Repo storages each project in its own dedicated repository

Monorepos

- A monorepo combines multiple projects or components into a single shared repository.

- A Multi-Repo storages each project in its own dedicated repository



Monorepos

- Create a repository where the monorepo will be allocated in.

Monorepos

- Create a repository where the monorepo will be allocated in.

- Add the repositories with:

```
$ git remote add <name> <HTTP link>
$ git fetch —-all
$ git subtree add --prefix=src/<directory-name> <name>
main —-squash
```

Monorepos

- Create a repository where the monorepo will be allocated in.

- Add the repositories with:

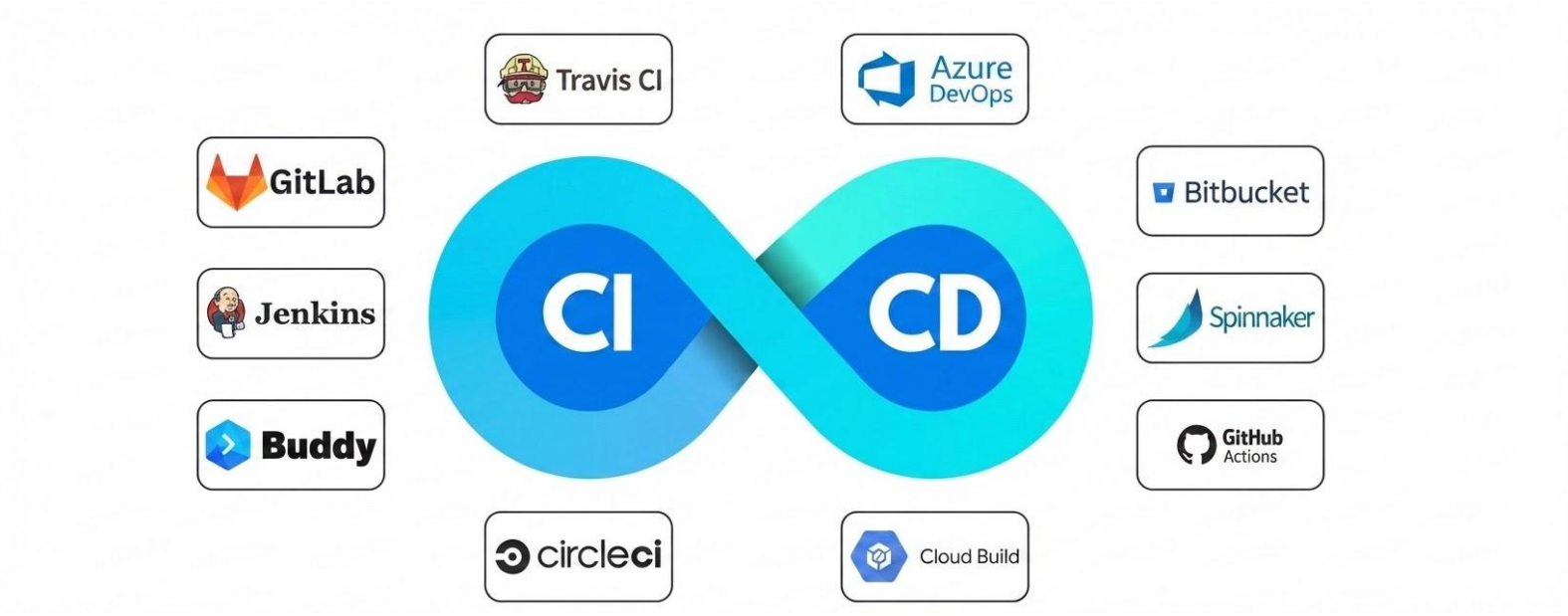
```
$ git remote add <name> <HTTP link>
$ git fetch —-all
$ git subtree add --prefix=src/<directory-name> <name>
main —-squash
```

- Configure `package.json` and `vite.config.ts`

Let’s see an example: PAI-Monorepo

CI/CD Alternatives

- Some popular alternatives to GitHub Actions:
  - Jenkins
  - GitLab CI/CD
  - CircleCI
  - Google Cloud Run



CI/CD Alternatives - Jenkins

- Some key characteristics of Jenkins:
  - Open-source automation server

Jenkins

CI/CD Alternatives - Jenkins

- Some key characteristics of Jenkins:
  - Open-source automation server
  - Self-hosted (runs on your own machine)

Jenkins

CI/CD Alternatives - Jenkins

- Some key characteristics of Jenkins:
  - Open-source automation server
  - Self-hosted (runs on your own machine)
  - Highly customizable (plugins)

Jenkins

CI/CD Alternatives - Jenkins

- Some key characteristics of Jenkins:
  - Open-source automation server
  - Self-hosted (runs on your own machine)
  - Highly customizable (plugins)
  - Supports many programming languages

Jenkins

CI/CD Alternatives - Jenkins

- Some key characteristics of Jenkins:
  - Open-source automation server
  - Self-hosted (runs on your own machine)
  - Highly customizable (plugins)
  - Supports many programming languages

- Problems
  - Complex setup

Jenkins

CI/CD Alternatives - Jenkins

- Some key characteristics of Jenkins:
  - Open-source automation server
  - Self-hosted (runs on your own machine)
  - Highly customizable (plugins)
  - Supports many programming languages

- Problems
  - Complex setup
  - Requires maintenance

Jenkins

CI/CD Alternatives - GitLab CI/CD

- Some key characteristics:
  - Integrated into GitLab repositories

CI/CD Alternatives - GitLab CI/CD

- Some key characteristics:
  - Integrated into GitLab repositories
  - Uses `.gitlab-ci.yml`

CI/CD Alternatives - GitLab CI/CD

- Some key characteristics:
  - Integrated into GitLab repositories
  - Uses `.gitlab-ci.yml`
  - Full DevOps lifecycle support

CI/CD Alternatives - GitLab CI/CD

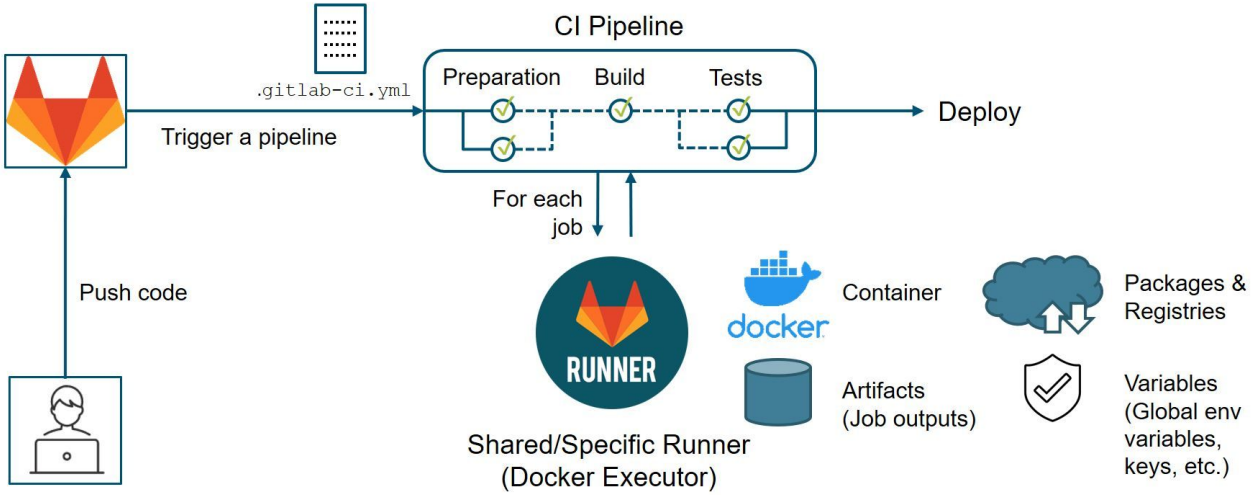
- Some key characteristics:
  - Integrated into GitLab repositories
  - Uses `.gitlab-ci.yml`
  - Full DevOps lifecycle support

- Advantages
  - All-in-one platform

CI/CD Alternatives - GitLab CI/CD

- Some key characteristics:
  - Integrated into GitLab repositories
  - Uses `.gitlab-ci.yml`
  - Full DevOps lifecycle support

- Advantages
  - All-in-one platform
  - Easy integration



CI/CD Alternatives - CircleCI

- Some key characteristics:
  - Cloud-based CI/CD

CI/CD Alternatives - CircleCI

- Some key characteristics:
  - Cloud-based CI/CD
  - Fast pipelines (parallel execution)

CI/CD Alternatives - CircleCI

- Some key characteristics:
  - Cloud-based CI/CD
  - Fast pipelines (parallel execution)
  - Simple YAML configuration

CI/CD Alternatives - CircleCI

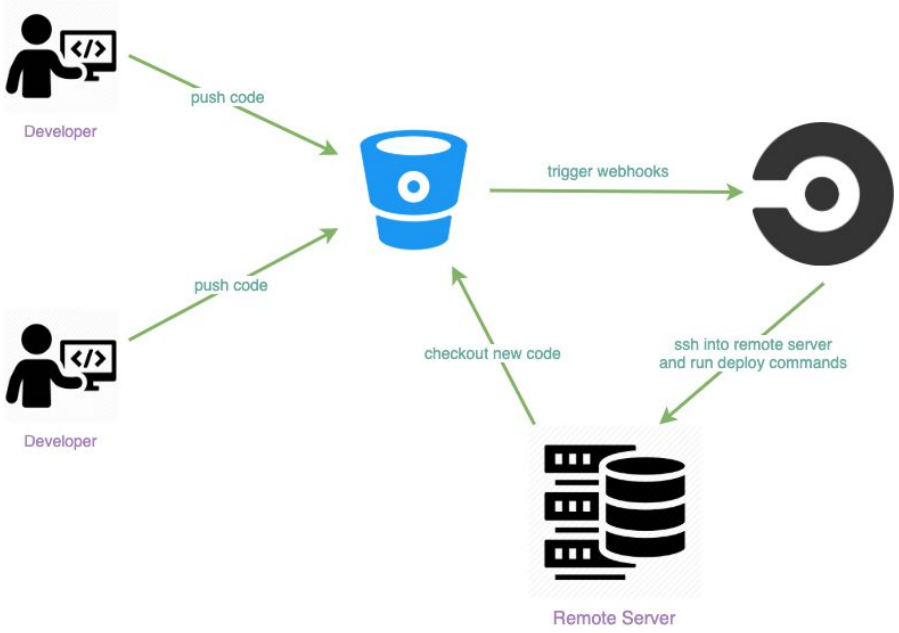
- Some key characteristics:
  - Cloud-based CI/CD
  - Fast pipelines (parallel execution)
  - Simple YAML configuration

- Advantages
  - Easy to use

CI/CD Alternatives - CircleCI

- Some key characteristics:
  - Cloud-based CI/CD
  - Fast pipelines (parallel execution)
  - Simple YAML configuration

- Advantages
  - Easy to use
  - High performance



CI/CD Alternatives - Google Cloud

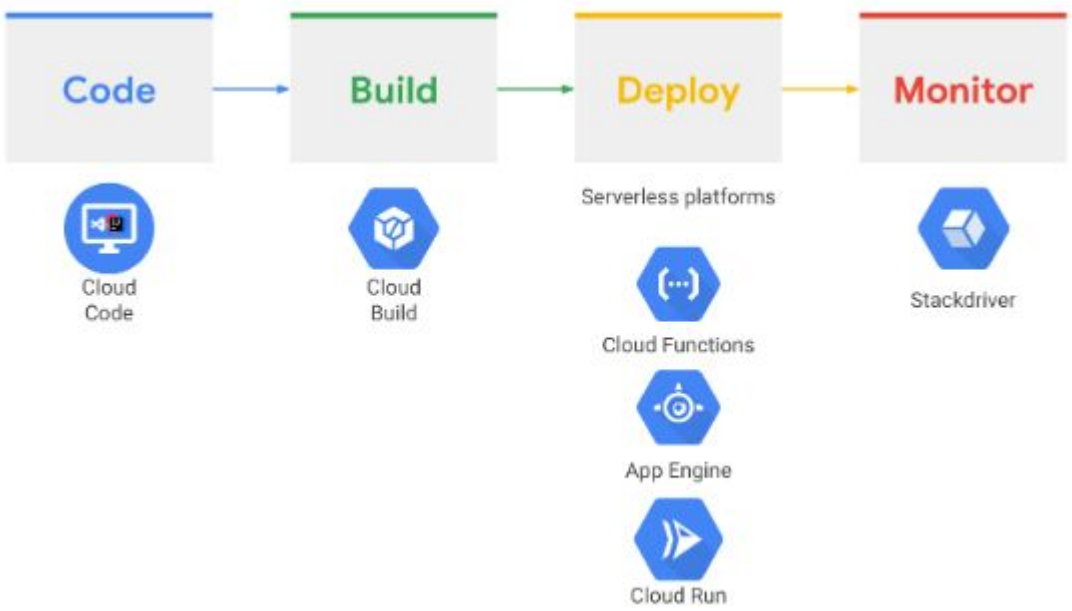
- Some key characteristics:
  - Paid cloud platform

CI/CD Alternatives - Google Cloud

- Some key characteristics:
  - Paid cloud platform
  - Runs containerized applications

CI/CD Alternatives - Google Cloud

- Some key characteristics:
  - Paid cloud platform
  - Runs containerized applications
  - Used for deployment, not only CI

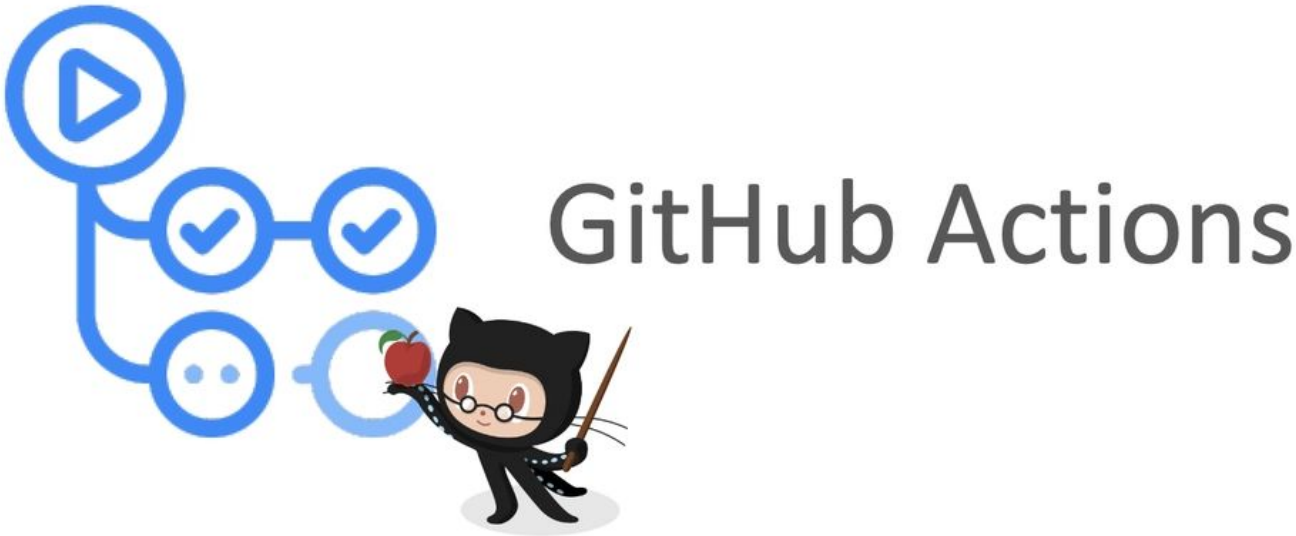


CI/CD Alternatives - Comparison

Tool	Type	Setup	Flexibility
Github Actions	Cloud	Easy	Medium
Jenkins	Self-hosted	Hard	High
GitLab CI/CD	Integrated	Medium	High
CircleCI	Cloud	Easy	Medium
Google Cloud	Cloud (Paid)	Medium	High

So, why GitHub Actions?

- Native GitHub integration
- No installation required
- Free tier available
- Simple workflows with YAML



References

- Learn Continuous Integration and Delivery with GitHub Actions, Docker, and Google Cloud Run
- GitHub Actions
- GitHub Actions TUTORIAL
- Tutorial Vite.js
- Setting up a TypeScript project with Vite
- Step-by-Step Guide: Deploy Your Vite Project to GitHub Pages with gh-pages

Universidad
de La Laguna